

GenAI Engineer - Content Systems | Take-Home Assessment



Assessment Overview

Role: GenAI Engineer - Content Systems

Submission Format: GitHub repo + Document + Video demonstration

Target Audience: 12th-grade graduate from India, limited English vocabulary, non-English-medium background, wants to kickstart an AI career

Context: This role isn't "write a prompt that generates content." It's building and owning an agentic system that generates learning content *and* judges its own quality — deciding whether that content is good enough to ship — the same loop you'd run on the job.



Build a Self-Evaluating Lesson Content Generator

 Build an agentic system that generates a beginner lesson for a given topic, evaluates it against a rubric you design, and regenerates it until it's good enough to ship. The point isn't one clever prompt — it's the full generate → evaluate → regenerate loop that decides, on its own, whether the content clears the bar before a human ever sees it.

Topic for submission: Introduction to RAG

INPUT → a topic, e.g. "RAG (Retrieval-Augmented Generation)"
— assume the learner starts from zero.

GENERATE → a standalone beginner lesson that teaches the topic — someone with no background should finish it and get it. Cover what it is, why it matters, and how it works.

EVALUATE → judge the lesson against a rubric of hard pass/fail checkpoints you design — no partial credit, each check clearly passes or fails. Dimensions worth weighing:

- accurate & grounded
- teaches by example
- covers the key points
- beginner-friendly language
- clear, no unexplained jargon
- coherent teaching flow

REGENERATE → on any fail, feed the reasons back and regenerate (max 1–2 retries, so the loop always terminates).

OUTPUT → the passing lesson + a rejection log — what failed, why, and what you changed on retry.

What we're looking for: more than a polished lesson, we want to see *how you build the system* — your planning, a robust architecture, and the reasoning behind your design choices (why this structure, these components, this evaluator, these trade-offs).

Across the Pipeline

SELF-EVOLVING → learn from repeated failures to sharpen prompts/rubrics

MEMORY → persists across runs; learns from feedback + logs

STACK → n8n / LangGraph / LangChain / Python+API · any model

Submission Guidelines

After completing the assignment, submit your outputs via the provided form:

1. **GitHub repo link** — your agentic workflow + README (setup & run instructions)
2. **Document link** (Google Doc / Notion) — the final lesson content created
3. **Loom video** (15–20 min) covering:
 - your agentic workflow running end-to-end
 - your evaluator catching a deliberate error
 - a face-visible walkthrough of how your workflow works

Submission form: <https://forms.gle/a7MJUNoTxvSdRB8R6>

All the best!